МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине «Аппаратное и программное обеспечение сетей» за 4 семестр

Тема: «Организация UDP – сервера и UDP – клиента»

Вариант 5

Выполнил:
студент 3 курса
группы ИИ-28
(1 подгруппа)
Деркач Е. В.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: изучить Winsock2 и написать UDP -сервер и UDP -клиент, которые обрабатывают поток символов группами по 64 и рвут связь, если в группе меньше 3 разных символов.

Ход работы

Постановка задачи:

Задания для реализации UDP сервера: после приема каждой группы из 64 символов от клиента сервер формирует и отсылает клиенту цепочку <«символ1-количество1», «символ2-количество2» и т.д.>, где «символ» - встреченный в группе символ, а «количество» - их количество в группе. Если в очередной группе число разных символов меньше 3, то отсылается соответствующее сообщение клиенту и сервер разрывает соединение.

Задания для реализации UDP клиента:

Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: **disconnect <адрес> <порт>**	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: **connect** **<адрес> <порт>** 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
PgDn	1), 2), 3)	-	2)

Код программы:

server.cpp

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS

#include <stdio.h>
#include <string.h>
#include <winsock2.h>
#include <windows.h>
#include <unordered_map>
#include <string>

#pragma comment(lib, "ws2_32.lib")

#define MY_PORT 666
#define GROUP_SIZE 64
#define BUFFER_SIZE 20480
```

```
struct ClientState
{
    char group[GROUP_SIZE];
    int filled;
    int stopped;
};

std::unordered_map<std::string, ClientState> clients;

std::string client_key(const sockaddr_in& addr)
{
    return std::string(inet_ntoa(addr.sin_addr)) + ":" +
           std::to_string(ntohs(addr.sin_port));
}

void send_message(SOCKET sock, const sockaddr_in& addr,
                  const char* message)
{
    sendto(sock, message, (int)strlen(message), 0,
           (const sockaddr*)&addr, sizeof(addr));
}

void process_group(SOCKET sock, const sockaddr_in& addr,
                   ClientState& state)
{
    int counts[256] = {};
    int order[256];
    int unique_count = 0;

    for (int i = 0; i < GROUP_SIZE; i++)
    {
        unsigned char ch = (unsigned char)state.group[i];

        if (counts[ch] == 0)
            order[unique_count++] = ch;

        counts[ch]++;
    }

    char outmsg[4096];

    if (unique_count < 3)
    {
        strcpy_s(outmsg, sizeof(outmsg),
            "ERROR: Less than 3 distinct characters. Session ended.");

        send_message(sock, addr, outmsg);

        state.filled = 0;
        state.stopped = 1;

        printf("%s\n", outmsg);
        return;
    }

    strcpy_s(outmsg, sizeof(outmsg), "<");
```

```
    for (int i = 0; i < unique_count; i++)
    {
        unsigned char ch = (unsigned char)order[i];
        char piece[64];

        if (ch >= 32 && ch <= 126 && ch != '\'' && ch != '\\')
        {
            sprintf_s(piece, sizeof(piece),
                "%s'%c-%d'",
                i ? ", " : "",
                ch,
                counts[ch]);
        }
        else
        {
            sprintf_s(piece, sizeof(piece),
                "%s0x%02X-%d",
                i ? ", " : "",
                ch,
                counts[ch]);
        }

        strcat_s(outmsg, sizeof(outmsg), piece);
    }

    strcat_s(outmsg, sizeof(outmsg), ">");

    send_message(sock, addr, outmsg);

    printf("Group processed: %s\n", outmsg);

    state.filled = 0;
}

int main()
{
    printf("UDP SERVER\n\n");

    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData))
    {
        printf("WSAStartup error %d\n", WSAGetLastError());
        return 1;
    }

    SOCKET mysocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (mysocket == INVALID_SOCKET)
    {
        printf("Socket error %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }
```

```
sockaddr_in local_addr = {};

local_addr.sin_family = AF_INET;
local_addr.sin_port = htons(MY_PORT);
local_addr.sin_addr.s_addr = htonl(INADDR_ANY);

if (bind(mysocket, (sockaddr*)&local_addr,
         sizeof(local_addr)) == SOCKET_ERROR)
{
    printf("Bind error %d\n", WSAGetLastError());

    closesocket(mysocket);
    WSACleanup();

    return 1;
}

printf("Waiting for UDP datagrams, port %d\n", MY_PORT);

char buff[BUFFER_SIZE];

while (true)
{
    sockaddr_in client_addr = {};
    int client_addr_size = sizeof(client_addr);

    int bytes_recv = recvfrom(
        mysocket,
        buff,
        sizeof(buff),
        0,
        (sockaddr*)&client_addr,
        &client_addr_size
    );

    if (bytes_recv == SOCKET_ERROR)
    {
        printf("recvfrom error %d\n", WSAGetLastError());
        continue;
    }

    std::string key = client_key(client_addr);

    printf("Received from %s: ", key.c_str());

    fwrite(buff, 1, bytes_recv, stdout);
    printf("\n");

    std::string message(buff, bytes_recv);

    if (message == "QUIT")
    {
        clients.erase(key);

        send_message(mysocket, client_addr, "Session ended.");
```

```
            printf("Session reset for %s\n", key.c_str());
            continue;
        }

        ClientState& state = clients[key];

        if (state.stopped)
            continue;

        for (int i = 0; i < bytes_recv; i++)
        {
            state.group[state.filled++] = buff[i];

            if (state.filled == GROUP_SIZE)
            {
                process_group(mysocket, client_addr, state);

                if (state.stopped)
                    break;
            }
        }
    }

    closesocket(mysocket);
    WSACleanup();

    return 0;
}
```

client.cpp

```
#define _WINSOCK_DEPRECATED_NO_WARNINGS

#include <stdio.h>
#include <string.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

#pragma comment(lib, "ws2_32.lib")

#define PORT 666
#define SERVERADDR "127.0.0.1"

SOCKET g_sock = INVALID_SOCKET;
FILE* g_log = NULL;
volatile LONG g_connected = 0;
CRITICAL_SECTION g_log_lock;

void log_event(const char* what, const char* detail)
{
    if (!g_log)
```

```
        return;

    EnterCriticalSection(&g_log_lock);

    time_t t;
    time(&t);

    struct tm tm_info;
    localtime_s(&tm_info, &t);

    char ts[64];

    sprintf_s(ts, sizeof(ts),
        "%04d-%02d-%02d %02d:%02d:%02d",
        tm_info.tm_year + 1900,
        tm_info.tm_mon + 1,
        tm_info.tm_mday,
        tm_info.tm_hour,
        tm_info.tm_min,
        tm_info.tm_sec);

    fprintf(g_log, "%s %s %s\n",
        ts, what, detail ? detail : "");

    fflush(g_log);

    LeaveCriticalSection(&g_log_lock);
}

DWORD WINAPI RecvThread(LPVOID)
{
    char buff[4096];

    while (InterlockedCompareExchange(&g_connected, 1, 1))
    {
        int n = recv(g_sock, buff, sizeof(buff) - 1, 0);

        if (n == SOCKET_ERROR)
        {
            if (InterlockedCompareExchange(&g_connected, 1, 1))
            {
                printf("\nReceive error: %d\n", WSAGetLastError());
                log_event("RECEIVE ERROR", "");
            }
            break;
        }

        if (n == 0)
            continue;

        buff[n] = 0;

        printf("\nS=>C: %s\n", buff);

        log_event("RECEIVED", buff);
```

```
        if (strncmp(buff, "ERROR:", 6) == 0)
        {
            InterlockedExchange(&g_connected, 0);
            break;
        }
    }

    return 0;
}

int main()
{
    printf("UDP CLIENT\n\n");
    printf("Auto-start with server %s:%d\n", SERVERADDR, PORT);
    printf("Type text and send with PgDn or Enter.\n");
    printf("Type quit or exit and press PgDn or Enter to stop.\n\n");

    InitializeCriticalSection(&g_log_lock);

    g_log = fopen("udp-client.log", "a");

    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
    {
        printf("WSAStartup error %d\n", WSAGetLastError());

        if (g_log)
            fclose(g_log);

        DeleteCriticalSection(&g_log_lock);
        return 1;
    }

    g_sock = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    if (g_sock == INVALID_SOCKET)
    {
        printf("Socket error %d\n", WSAGetLastError());

        WSACleanup();

        if (g_log)
            fclose(g_log);

        DeleteCriticalSection(&g_log_lock);
        return 1;
    }

    sockaddr_in dest_addr = {};

    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(PORT);
    dest_addr.sin_addr.s_addr = inet_addr(SERVERADDR);

    if (connect(g_sock, (sockaddr*)&dest_addr, sizeof(dest_addr))
```

```
    == SOCKET_ERROR)
{
    printf("Server address error %d\n", WSAGetLastError());

    closesocket(g_sock);
    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);
    return 1;
}

printf("UDP communication target: %s:%d\n",
    SERVERADDR, PORT);

log_event("CLIENT START", SERVERADDR ":666");

InterlockedExchange(&g_connected, 1);

HANDLE recvThread = CreateThread(
    NULL, 0, RecvThread, NULL, 0, NULL);

if (!recvThread)
{
    printf("CreateThread error %lu\n", GetLastError());

    InterlockedExchange(&g_connected, 0);
    closesocket(g_sock);
    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);
    return 1;
}

char line[1024];
int len = 0;

printf("S<=C: ");
fflush(stdout);

while (InterlockedCompareExchange(&g_connected, 1, 1))
{
    int c = _getch();

    if (c == 27)
        break;

    if (c == 0 || c == 224)
    {
        int k = _getch();
```

```
    if (k == 81 && len > 0)
    {
        line[len] = 0;

        if (!strcmp(line, "quit") || !strcmp(line, "exit"))
            break;

        int sent = send(g_sock, line, len, 0);

        if (sent == SOCKET_ERROR)
        {
            printf("\nSend error %d\n", WSAGetLastError());
            break;
        }

        log_event("SENT", line);

        printf("\nSent %d chars.\nS<=C: ", sent);

        len = 0;
        fflush(stdout);
    }

    continue;
}

if (c == '\r' || c == '\n')
{
    if (len > 0)
    {
        line[len] = 0;

        if (!strcmp(line, "quit") || !strcmp(line, "exit"))
            break;

        int sent = send(g_sock, line, len, 0);

        if (sent == SOCKET_ERROR)
        {
            printf("\nSend error %d\n", WSAGetLastError());
            break;
        }

        log_event("SENT", line);

        printf("\nSent %d chars.\nS<=C: ", sent);

        len = 0;
        fflush(stdout);
    }

    continue;
}

if (c == 8)
{
```

```
            if (len > 0)
            {
                len--;
                printf("\b \b");
                fflush(stdout);
            }

            continue;
        }

        if (c >= 32 && len < (int)sizeof(line) - 1)
        {
            line[len++] = (char)c;
            putchar(c);
            fflush(stdout);
        }
    }

    if (InterlockedCompareExchange(&g_connected, 1, 1))
    {
        const char* quitMessage = "QUIT";
        send(g_sock, quitMessage, (int)strlen(quitMessage), 0);
    }

    InterlockedExchange(&g_connected, 0);

    shutdown(g_sock, SD_BOTH);
    closesocket(g_sock);

    WaitForSingleObject(recvThread, 1000);
    CloseHandle(recvThread);

    log_event("CLIENT END", "");

    WSACleanup();

    if (g_log)
        fclose(g_log);

    DeleteCriticalSection(&g_log_lock);

    return 0;
}
```

Тестирование программы:

```
PS D:\Course 3\aipos> g++ server.cpp -o server.exe -lws2_32
PS D:\Course 3\aipos> .\server
SERVER

Waiting for connections, port 666
+genuine-software.autodesk.com [127.0.0.1] new connect!
1 user on-line
group processed: <'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1', 't-2', 'i-1', 'r-1'>
-disconnect
No User on line
```

Problems Output Debug Console Terminal Ports powershell

```
PS D:\Course 3\aipos> g++ client.cpp -o client.exe -lws2_32
PS D:\Course 3\aipos> .\client
CLIENT

Auto-connect to 127.0.0.1:666
Type text, send - PgDn or Enter, exit - quit/exit + PgDn or Enter

Connection to 127.0.0.1 established successfully
S<=C:S=>C:Hello, Student!
aaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbcccccccccc
sent 45 chars.
S<=C:aabavbvvadfjhltidrt
sent 19 chars.
S<=C:S=>C:<'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1', 't-2', 'i-1', 'r-1'>
aaaaaaaaaaaaaaasssssssssssssssddddddddddddddfffffffffffff
sent 55 chars.
S<=C:exit
Connection closed by server
PS D:\Course 3\aipos>
```

tcp-client.log

```
2026-09-22 12:04:59 CONNECTION START 127.0.0.1:666
2026-09-22 12:04:59 RECEIVED Hello, Student!


2026-09-22 12:05:05 SENT aaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbcccccccccc
2026-09-22 12:05:09 SENT aabavbvvadfjhltidrt
2026-09-22 12:05:09 RECEIVED <'a-19', 'b-23', 'c-9', 'v-3', 'd-2', 'f-1', 'j-1', 'h-1', 'l-1',
't-2', 'i-1', 'r-1'>


2026-09-22 12:05:17 SENT aaaaaaaaaaaaaaasssssssssssssssddddddddddddddfffffffffffff
2026-09-22 12:05:21 CONNECTION END
2026-09-22 12:05:21 CONNECTION END
```

Вывод: изучил функции Winsock2 (WSAStartup, socket, bind, listen, accept, connect, send, recv, closesocket, WSACleanup); написал и отладил сервер и клиент для варианта 5; сервер считает вхождения символов в каждой группе из 64 и разрывает соединение при <3 разных символах, клиент автоматически подключается к 127.0.0.1:666, отправляет строки по PgDn/Enter и ведёт лог udp-client.log; программы проверены, задание выполнено полностью.